

FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Semestrální práce KIV/PC

Algoritmus Minimax v deskové hře Othello

Student: Tomáš Tobrman
Osobní číslo: A22B0199P
Email: ttobrman@students.zcu.cz
Datum: 14. ledna 2024

Obsah

1 Zadání **3**

2 Analýza úlohy **3**
2.1 Pravidla hry . . . 3
2.2 Reprezentace hrací plochy . . . 4
2.3 Ověření tahu . . . 5
2.4 Minimax algoritmus . . . 5
2.4.1 Evaluace stavu hry . . . 5
2.4.2 Alpha-Beta prořezávání . . . 5

3 Popis implementace **6**
3.1 Obsah souboru `include/othello/types.h` . . . 6
3.1.1 Datový typ `bool` . . . 6
3.1.2 Hrací plocha `board_t` . . . 6
3.1.3 Historie `history_t` . . . 7
3.1.4 Informace hráče `player_t` . . . 7
3.1.5 Nastavení a stav aplikace `settings_t` . . . 7
3.2 Výpočet možných tahů . . . 8
3.3 Historie hry . . . 9
3.4 Minimax algoritmus . . . 10

4 Uživatelská příručka **12**
4.1 Sestavení . . . 12
4.2 Spuštění . . . 13
4.3 Ovládání . . . 14
4.4 Struktura vstupního souboru . . . 14

5 Závěr **15**

Seznam obrázků

1 Výchozí stav hrací plochy Othello 3
2 Možné tahy z počátečního stavu 4
3 Zabrání protějšího kamene 4
4 Rozhodovací strom vytvořený algoritmem **Minimax** s vyhledávací hloubkou 2. Hráč s bílými kameny je maximalizující, hráč s černými kameny je minimalizující. . 5

Seznam tabulek

1 Popis přepínačů a jejich argumentů. 13
2 Popis příkazů lidského hráče. 14
3 Struktura možných nastavení hry pomocí vstupního souboru. 14

Seznam konzolí

1 Vlastní implementace logického datového typu `bool`. 6
2 Struktura hrací plochy `board_t`. 6
3 Struktura historie `history_t`. 7
4 Struktura informací o hráči `player_t`. 7
5 Struktura nastavení a stavu aplikace `settings_t`. 8
6 Cyklus pro kontrolu tahů v modulu `src/board.c`. 8
7 Kontrola herních polí ve 3×3 rozmezí. 9
8 Otočení protějších kamenů. 9
9 Přidání příkazu do historie. 10
10 Část algoritmu **Minimax** pro maximalizujícího hráče. 10
11 Část algoritmu **Minimax** pro minimalizujícího hráče. 11
12 Statická evaluace stavu hry. Tato část kódu se nachází na začátku funkce. 11
13 Instalace nástrojů přes manažera pracovních balíků Chocolatey. 12
14 Ověření existence nástrojů `make.exe` a `gcc.exe` z příkazové řádky. 12
15 Ukázka sestavení projektu pomocí nástroje `make` na operačním systému **Mac OS X**. 13
16 Ukázka spuštění programu v operačním systému Windows (kvůli limitaci dokumentace nelze ukázat celý výstup). 13
17 Ukázkový vstupní soubor. 14

1 Zadání

Naprogramujte v jazyce ANSI C přenositelnou **konzolovou aplikaci**, která umožní lidským uživatelům či umělým inteligencím změřit si své síly v deskové hře **Othello**. Vámi implementovaná umělá inteligence bude pro zjištění optimálního tahu využívat algoritmus **Minimax** rozšířený o tzv. **Alfa-beta prořezávání**. Jednotlivé hry bude možné uložit a případně později načíst v definovaném formátu do textového souboru. Podoba konzolového rozhraní je plně ve vaší kompetenci.

Kompletní verzi zadání [2] můžete najít na této webové adrese:
`https://www.kiv.zcu.cz/studies/predmety/pc/data/works/sw2023-03.pdf`

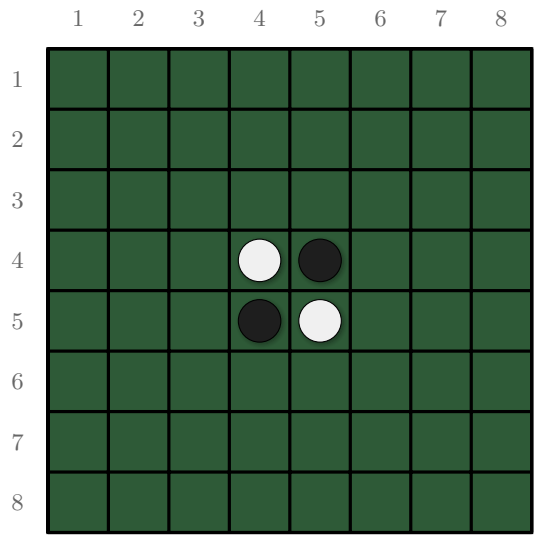
2 Analýza úlohy

Tato kapitola bude poukazovat na různá řešení problémů, které se ukázaly při tvorbě funkční deskové hry **Othello** s implementací algoritmu **Minimax** a jeho rozšíření **Alpha-Beta prořezávání** v jazyce **ANSI C**.

2.1 Pravidla hry

Pravidla deskové hry **Othello** se zdají jednoduchá, ale stále nemusí být triviální implementovat v programovacím jazyce. Tato sekce probere pravidla pro vykonání tahu a konec hry.

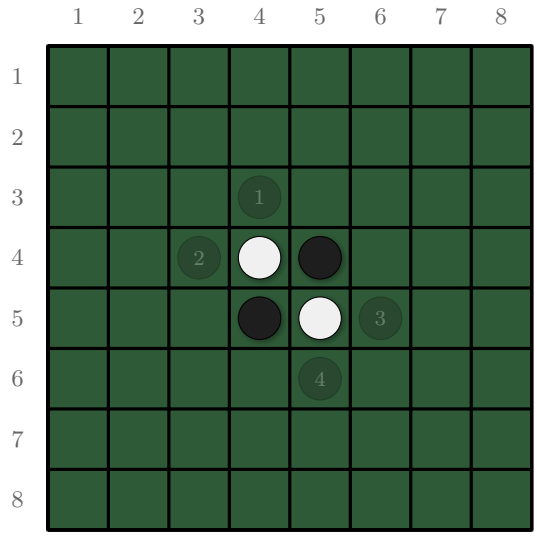
Hrací plocha má podobu 8×8 čtvercových polí uskupených na desku (Obrázek 1, strana 3). V počátečním stavu hry se položí 4 kameny doprostřed hrací plochy tak, že na polích [4;4] a [5;5] je viditelná část kamenů bílá a zbylé dva kameny mají viditelnou barvu černou.



Obrázek 1: Výchozí stav hrací plochy Othello

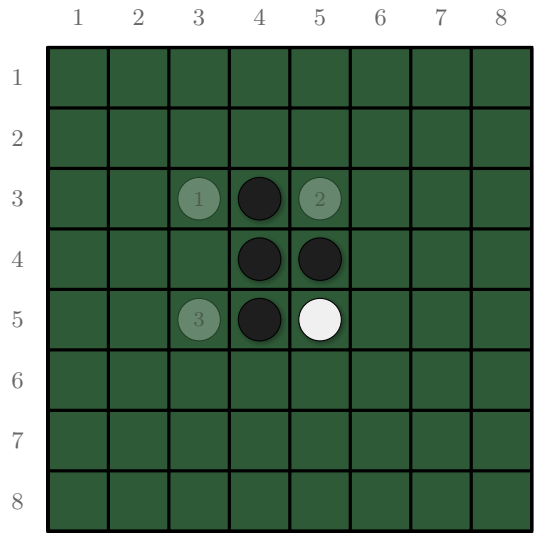
Narozdíl od šachů nebo dámy, ve hře Othello začíná hráč s *černými* kameny. Tah se dá provést, pokud hráč v řadě obklíčí kameny protihráče. Toto může být v jakémkoliv z osmi směrů. Obklíčené kameny se obrátí na barvu hráče, který tento tah provedl. Pokud by tah nezajmul protější kameny, není validním tahem a hráč musí vybrat jiný. Dalším možným tahem je přeskočení svého kola. Toto je ale možné pouze v případě, kdy nelze odehrát nic jiného.

V počátečním stavu jsou čtyři možné tahy, znázorněné na obrázku 2 na straně 4 průhlednými kameny. Tahy jsou číslovány v pořadí zleva doprava a shora dolů.



Obrázek 2: Možné tahy z počátečního stavu

Zvolí-li hráč tah číslo 1 (Obrázek 3, strana 4), zabere protější kámen na pozici [4;4] a přetočí ho na svou barvu. Dále hraje protější hráč.



Obrázek 3: Zabrání protějšího kamene

Hráči se střídají ve hraní dokud ani jeden hráč nemůže udělat tah nebo celá plocha je zabrána kameny. Hráč s vyšším počtem kamenů na ploše vyhrává.

2.2 Reprezentace hrací plochy

Hrací deska se díky pevné a opakující se struktuře dá jednoduše v programu reprezentovat jako dvourozměrná matice, či jednorozměrné pole, kde kameny jsou reprezentovány číselnou hodnotou podle toho, ke kterému z hráčů patří. Toto jednoduché řešení umožňuje pomocí dvourozměrných souřadnic čtení a změnu jednotlivých polí hrací plochy v $O(1)$ složitosti.

Rozdíl mezi jednorozměrným polem a dvourozměrnou maticí je adresování jednotlivých prvků.

U jednorozměrného pole, resp. dvourozměrné matice, musíme použít jednorozměrný index, resp. dvourozměrné souřadnice. Výběr záleží pouze na vlastní preferenci, jelikož je možný převod mezi oběma způsoby adresování. Dvourozměrné souřadnice se převedou na jednorozměrný index pomocí $i = y \cdot w + x$, kde i je výsledný index, $[x; y]$ jsou komponenty dvourozměrných souřadnic s počátkem $[0; 0]$ a w je šířka hracího pole. Jednorozměrný index se převádí na dvourozměrné souřadnice s počátkem $[0; 0]$ se stejnými proměnnými přes $x = i \mod w$ a $y = i/w$.

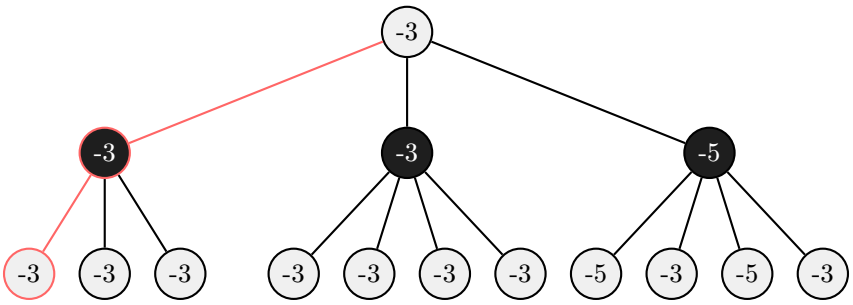
2.3 Ověření tahu

Kontrola se může provádět při pokusu o tah, kde se ověří pouze jedna pozice. Další možností je provést kontrolu všech polí před zahájením tahu a uložit jejich výsledek do pole či matice. Pokud se počítá s nejhorším případem, kde hráč bude vybírat zásadně neplatná pole a toto provede víckrát, než je polí na hrací desce, pak je výhodnější vše vypočítat předem. V reálné aplikaci však k tomuto většinou nedojde a rychlejší by tedy byla kontrola teprve při pokusu o tah.

2.4 Minimax algoritmus

Algoritmus **Minimax** je jednoduchý algoritmus pro vyhodnocování nejlepšího tahu. K tomuto uvažuje, že protější hráč dělá totéž. Tedy, i když pro **Minimax** nebude momentální tah výhodný, zamezí tím alespoň protějšímu hráči odehrát svůj nejlepší tah.

Minimax funguje na principu rozhodovacího stromu (Obrázek 4, strana 5). Každý uzel představuje stav hry, do kterého se program dostane, bude-li následovat cestu k němu. Nejpřirozenější implementací takové struktury je **rekurze**, kde se prvně vyhodnotí maximální hloubka stromu a výsledek se přenáší zpětně až do kořene. Ten následně dokáže vyhodnotit, který z příslušných tahů vede k nejlepšímu stavu.



Obrázek 4: Rozhodovací strom vytvořený algoritmem **Minimax** s vyhledávací hloubkou 2. Hráč s bílými kameny je maximalizující, hráč s černými kameny je minimalizující.

2.4.1 Evaluace stavu hry

Implementace samotného vyhodnocení stavů je na každém programu a může se značně lišit mezi typy her. V případě hry **Othello** lze využít jednoduché ohodnocení, kde se pouze porovná počet vlastních kamenů s těmi protihráče. Některé pozice jsou však výhodnější než jiné. Například, pokud hráč ovládá okraje hrací plochy, pak má tyto pozice zajištěny pro sebe, jelikož není jak obklíčit takové kameny. Nejlepšími pozicemi jsou rohy desky, kde je nemožné takový kámen zabrat protihráčem. Naopak tyto kameny dále pomáhají k otočení dalších kamenů.

2.4.2 Alpha-Beta prořezávání

Rozšíření algoritmu **Minimax**, zvané **Alpha-Beta prořezávání**, ho dokáže v průměru zrychlit, ale není tomu zaručeno. Rozšíření funguje na principu, kde není třeba provádět další výpočty v aktuální větvi stromu, jelikož algoritmus již nedostane lepší výsledek, než který je mu již přístupný.

Největší vliv na rychlost algoritmu by tedy rozšíření mělo, pokud by možné tahy byly již seřazeny od nejvíce do nejméně výhodných.

3 Popis implementace

Aplikace hry **Othello** s algoritmem **Minimax** byla implementována v jazyce **ANSI C 90** na operačním systému **Mac OS X 11.7.10 Big Sur**. Projekt je kompletně popsán dokumentačními komentáři a byl rozdělen do několika modulů pro rozdělení odpovědnosti a přehlednost zdrojového kódu.

3.1 Obsah souboru `include/othello/types.h`

Hlavičkový soubor `include/othello/types.h` obsahuje všechny typové definice (`typedef`), struktury (`struct`) a výčtové typy (`enum`). Tato podsekce probere typy a struktury nutné pro zbytek programu.

3.1.1 Datový typ `bool`

V projektu byl často využit vlastně definovaný logický datový typ `bool` (Konzole 1, strana 6), který v základním jazyce **ANSI C** neexistuje. Tento datový typ byl vytvořen kvůli přehlednosti, aby bylo jasné, co funkce vrací, namísto typu `int`, který může představovat velké množství hodnot.

```
/** Logical bool type implementation */
typedef enum {
    /* logical false */
    false = 0,
    /* logical true */
    true = 1
} bool;
```

Konzole 1: Vlastní implementace logického datového typu `bool`.

3.1.2 Hrací plocha `board_t`

```
/** Board structure holding the game data and available moves */
typedef struct {
    /* Number of board columns */
    unsigned int width;
    /* Number of board rows */
    unsigned int height;
    /* Size of data and moves arrays */
    unsigned int size;
    /* Player stones positions */
    player_color *data;
    /* Available moves */
    unsigned int *moves;
} board_t;
```

Konzole 2: Struktura hrací plochy `board_t`.

Hrací plocha hry **Othello** je v tomto programu reprezentována strukturou, která obsahuje všechny informace o její velikosti a datech (Konzole 2, strana 6) jednorozměrným polem o velikosti $w \cdot h$, kde w je šířka plochy, nebo-li počet sloupců (`unsigned int width`), a h je výška plochy, nebo-li

počet řádků (`unsigned int height`). Pole `player_color *data` obsahuje pozice všech kamenů. Pole `unsigned int *moves` ukazuje, na kterých pozicích je možné udělat validní tah.

3.1.3 Historie `history_t`

Struktura `history_t` (Konzole 3, strana 7) pouze existuje jako *obal* nad polem `char *`. Dalším prvkem této struktury je `size_t size`, což znázorňuje počet prvků v poli `char **commands`.

```
1 /** Command history */
2 typedef struct {
3     /* Array of commands */
4     char **commands;
5     /* Size of the commands array */
6     size_t size;
7 } history_t;
```

Konzole 3: Struktura historie `history_t`.

3.1.4 Informace hráče `player_t`

Do potřebných informací o hráči (Konzole 4, strana 7) patří barva kamenů, se kterými hraje (řádek 4), typ (řádek 6), jméno (řádek 12) a maximální hloubka algoritmu **Minimax** (řádek 16).

```
1  /** Player info */
2  typedef struct {
3      /* Player color */
4      player_color color;
5      /* If the player is played by a human or minimax */
6      player_type type;
7      /* String name of the player type */
8      char *type_str;
9      /* If the type has been set from the default value */
10     bool type_set;
11     /* Name of the player */
12     char name[INPUT_SIZE];
13     /* If the name has been set from the default value */
14     bool name_set;
15     /* Max depth of the minimax algorithm for his player */
16     int depth;
17     /* If the depth has been set from the default value */
18     bool depth_set;
19 } player_t;
```

Konzole 4: Struktura informací o hráči `player_t`.

3.1.5 Nastavení a stav aplikace `settings_t`

Veškeré nastavení a stav aplikace je obsaženo ve struktuře `settings_t` (Konzole 5, strana 8). Její funkce je sloužit jako seskupení velkého množství proměnných, které by jinak musely žít na zásobníku. Tyto hodnoty se často předávají dalším funkcím, takže je lepší předávat jeden ukazatel, který má přístup ke všemu. Struktura například obsahuje definice obou hráčů (řádky 4 a 6), vlastnosti hrací desky (řádky 8–16), vstupní a výstupní soubory (řádky 20 a 22) a zda má aplikace skončit (řádek 24).

```c
/** Application settings */
typedef struct {
    /* Black Player */
    player_t *player_black;
    /* White Player */
    player_t *player_white;
    /* Number of board columns */
    int board_cols;
    /* Number of board rows */
    int board_rows;
    /* If the board size has been set from the default value */
    bool size_set;
    /* Column of the initial top left stone */
    int init_col;
    /* Row of the initial top left stone */
    int init_row;
    /* If the initial position has been set from the default value */
    bool init_set;
    /* Output gamefile pointer */
    FILE *file_out;
    /* Input gamefile pointer */
    FILE *file_in;
    /* If the application should exit - ends the game loop */
    bool should_exit;
    /* If the players should be switched and a next move should be played */
    bool next_move;
} settings_t;
```

Konzole 5: Struktura nastavení a stavu aplikace `settings_t`.

3.2 Výpočet možných tahů

V tomto projektu jsem se rozhodl použít způsob výpočtu všech možných tahů předem. Výpočet teprve při pokusu o tah by byl teoreticky rychlejší a nebylo by zapotřebí přídavného pole ve struktuře `board_t` (Konzole 2, strana 6, řádek 12).

```c
    for (row = 0; row < board->height; ++row) {
        for (col = 0; col < board->width; ++col) {
            /* get index from 2D coordinates */
            i = row * board->width + col;

            /* skip cell if occupied */
            if (board->data[i]) {
                board->moves[i] = 0;
                continue;
            }

            /* check 3x3 area around cell */
            board->moves[i] = check_3x3(board, color, col, row, false);
        }
    }
```

Konzole 6: Cyklus pro kontrolu tahů v modulu `src/board.c`.

Funkce `void board_calculate(...)` v modulu `"othello/board.h"` vykoná výpočet možných tahů pro danou barvu kamenů `player_color color`. Výpočet tedy koná nad každým herním polem (Konzole 6, strana 8). Pokud pole není zabráno kamenem, pak začne kontrola okolo, kde se hledají kameny protějšího hráče.

```
    for (dir = 0; dir < 8; ++dir) {
        /* offset position in a direction */
        ocol = col + dir_col[dir];
        orow = row + dir_row[dir];

        /* out of bounds */
        if (ocol < 0 || ocol >= board->width || orow < 0 ||
            orow >= board->height)
            continue;

        /* get index from 2D coordinates */
        i = orow * board->width + ocol;

        /* found enemy stone, check in their direction */
        if (board->data[i] != color)
            weight += check_direction(board, color, col, row, dir_col[dir],
                                      dir_row[dir], flip);
    }
```

Konzole 7: Kontrola herních polí ve 3×3 rozmezí.

Je-li kámen protějšího hráče nalezen, algoritmus pokračuje v jeho směru. Narazí-li na prázdné místo nebo konec hrací desky, navrátí se hodnota 0, která značí, že v tomto směru lze sebrat 0 kamenů. Pokud ale je nalezen kámen hráče vykonávající tento tah, pak se protější kameny berou jako obklíčené a funkce `static int check_direction(...)` navrátí počet obklíčených kamenů v tomto směru.

Pokud parametr `bool flip` má hodnotu `true`, pak při nalezení validního tahu funkce `static int check_direction(...)` také *obrátí* protější kameny v opačném směru, dokud nenarazí na původní pozici kontroly (Konzole 8, strana 9).

```
            /* flip the stones while moving back to the original possition */
            while (flip) {
                if (ocol == col && orow == row) break;
                board->data[orow * board->width + ocol] = color;
                /* move back */
                ocol -= dir_col;
                orow -= dir_row;
            }
```

Konzole 8: Otočení protějších kamenů.

3.3 Historie hry

Vždy po úspěšném tahu nebo přeskočení tahu se použitý příkaz zapíše do historie, aby bylo možné jej následně uložit do výstupního souboru. Historie je implementována jako pole řetězců (sekce 3.1.3, strana 7).

Pro přidání prvku do historie je třeba ho nejdříve zkopírovat. Pokud by se uložil originální řetězec, pak by pole drželo adresu pouze tohoto jednoho řetězce a celá historie by byla přepsána posledním daným příkazem.

```
    /* create a new string to save into the history array */
    str = (char *)malloc(sizeof(char) * INPUT_SIZE);
    if (!str) return false;

    /* copy the command into the new string */
    str = strcpy(str, command);

    /* add the new string to history as a last element */
    history->commands[history->size] = str;
    ++history->size;
```

Konzole 9: Přidání příkazu do historie.

3.4 Minimax algoritmus

Implementace algoritmu **Minimax** s **Alpha-Beta prořezáváním** následovala pseudokód z **YouTube** videa od **Sebastiana Lague** [1], který stručně popsal jeho funkčnost. Jeho implementace využívá rekurzivní volání funkce algoritmu pro stavbu vyhledávacího stromu.

```
    /* calculate evaluation for maximizing player */
    if (maximizing) {
        ret_eval = INT_MIN;
        for (i = 0; i < board->size; ++i) {
            if (!board->moves[i]) continue;

            copy = board_copy(board);
            col  = (i % copy->width) + 1;
            row  = (i / copy->width) + 1;

            board_calculate(copy, color);
            if (board_can_play(board)) board_play(copy, color, col, row);
            board_calculate(copy, other_color);
            eval = minimax(color, copy, depth - 1, alpha, beta, false, 0, 0);
            board_free(&copy);

            if (eval > ret_eval) {
                ret_eval = eval;
                if (play_col) *play_col = col;
                if (play_row) *play_row = row;
            }

            alpha = alpha > eval ? alpha : eval;
            if (beta <= alpha) break;
        }
    }
```

Konzole 10: Část algoritmu **Minimax** pro maximalizujícího hráče.

Maximalizující hráč (Konzole 10, strana 10) porovnává aktuální evaluaci s maximální evaluací co dostal. Pokud aktualní evaluace stavu hry je větší než dosud maximální evaluace, tato hodnota se

nahradí novou a pokud existují ukazatele na hratelnou pozici, nastaví se také jejich nové hodnoty. Jako součást **Alpha-Beta prořezávání** se také porovnává hodnota `alpha`. Pokud je hodnota `alpha` momentálně větší než hodnota `beta` nebo je jí rovna, pak cyklus končí a neprovádí se v této větvi stromu další evaluace.

Minimalizující hráč (Konzole 11, strana 11) provádí podobné kroky jako ten maximalizující, ale porovnání je převrácené. Tentokrát se hledá nejnižší evaluace a nastavuje se hodnota `beta`. Znovu, pokud je hodnota `beta` menší než hodnota `alpha` nebo je jí rovna, pak cyklus končí a neprovádí se další evaluace v této větvi rozhodovacího stromu.

```
    /* calculate evaluation for minimazing player */
    else {
        ret_eval = INT_MAX;
        for (i = 0; i < board->size; ++i) {
            if (!board->moves[i]) continue;

            copy = board_copy(board);
            col  = (i % copy->width) + 1;
            row  = (i / copy->width) + 1;

            board_calculate(copy, other_color);
            if (board_can_play(board)) board_play(copy, other_color, col, row);
            board_calculate(copy, color);
            eval = minimax(color, copy, depth - 1, alpha, beta, true, 0, 0);
            board_free(&copy);

            if (eval < ret_eval) {
                ret_eval = eval;
                if (play_col) *play_col = col;
                if (play_row) *play_row = row;
            }

            beta = beta < eval ? beta : eval;
            if (beta <= alpha) break;
        }
    }
```

Konzole 11: Část algoritmu **Minimax** pro minimalizujícího hráče.

```
    /* static evaluation */
    if (depth <= 0) {
        for (i = 0; i < board->size; ++i) {
            if (board->moves[i]) {
                if (play_col) *play_col = (i % board->width) + 1;
                if (play_row) *play_row = (i / board->width) + 1;
                break;
            }
        }
        eval = board_count(board, color) - board_count(board, other_color);
        return eval;
    }
```

Konzole 12: Statická evaluace stavu hry. Tato část kódu se nachází na začátku funkce.

Při každé evaluaci stavu se vytvoří kopie struktury hrací desky `board_t` na které je následně proveden ověřovaný tah. Tato kopie je po rekurzivním volání hned uvolněna, jelikož není dále potřeba.

Pokud se algoritmus rekurzivním voláním dostane na hloubku 0, nebo na této hloubce začne, provede se statická evaluace stavu hry (Konzole 12, strana 11). Pokud existují ukazatele na hodnoty tahu `play_col` a `play_row`, pak jsou nastaveny na první možný tah, kterým bude pozice nejvíce vlevo nahoře. Následně se vypočte hodnota evaluace jako rozdíl počtů kamenů každé barvy na hrací desce. Tato hodnota je vrácena jako návratová hodnota funkce. Pokud byl tento stav dosažen při konstrukci rozhodovacího stromu, pak je tato hodnota použita při porovnání popsaných výše.

Nastavené hodnoty `play_col` a `play_row` jsou použity jako další návratové hodnoty funkce, kterými algoritmus dokáže odehrát zvolený tah. Pokud tah nebyl zvolen, pak zůstávají tyto proměnné s hodnotami 0 a jsou dále vyhodnoceny jako příkaz `pass` a tah hráče je přeskočen. Toto může nastat pokud neexistuje žádný možný tah, který by hráč mohl vykonat.

4 Uživatelská příručka

V této sekci je popsaný postup sestavení a spuštění aplikace `othello.exe` a příkazy pro hraní a ovládání programu.

4.1 Sestavení

Pro sestavení aplikace do spustitelné formy je zapotřebí programů `make` a `gcc`. Pro operační systémy **Windows** se oba programy nácházejí v prostředí **MinGW**. Operační systémy typu **Linux** by měly tyto nástroje mít již předinstalované. Pro ověření jejich existence je stačí vyvolat v konzolovém prostředí. Pokud jejich výstup vypadá podobně jako v ukázce konzole 14 na straně 12, pak jsou nástroje nainstalovány. Na operačním systému **Mac OS X** je třeba nainstalovat *Command Line Tools for Xcode*. Jejich instalaci lze vyvolat příkazem `gcc`, pokud nejsou již nainstalované.

Pro instalaci **MinGW** následujte pokyny na stránkách `https://www.mingw-w64.org/`, nebo využijte nástroj **Chocolatey** (Konzole 13, strana 12), pokud je nainstalovaný.

```
PS C:\> choco install mingw
```

Konzole 13: Instalace nástrojů přes manažera pracovních balíků Chocolatey.

Po nainstalování **MinGW** by měly být přístupné příkazy `make.exe` a `gcc.exe` v příkazové řádce. Lze využít program **CMD** nebo **PowerShell** k otevření konzole. Ověření správné instalace se dá provést jejich vyvoláním (Konzole 14, strana 12). Pokud výstup programů vypadá jinak, než v ukázce konzole, pak je možné, že je třeba přidat cestu k programům do proměnné prostředí `PATH` a restartovat konzoli.

```
PS C:\> gcc.exe
gcc.exe: fatal error: no input files
compilation terminated.
PS C:\> make.exe
make.exe: *** No targets specified and no makefile found. Stop.
```

Konzole 14: Ověření existence nástrojů `make.exe` a `gcc.exe` z příkazové řádky.

Jestli že máte dostupné oba nástroje z příkazové řádky, navigujte se do adresáře s touto semestrální prací příkazem `cd`. Na **Windows** využijte příkaz `make.exe -f Makefile.win` a na systémech **Linux** a **Mac OS X** příkaz `make -f Makefile` nebo jen `make`. Pokud vše proběhlo správně,

výstup by měl vypadat podobně jako v ukázce 15 na straně 13. Spustitelný program by se nyní měl nacházet v kořenové složce projektu.

```
user@machine:~$ cd ~/semestralka
user@machine:~/semestralka$ make -f Makefile
mkdir -p obj
gcc -Wall -xc -ansi -pedantic -Iinclude -g -c -o obj/history.o src/history.c
gcc -Wall -xc -ansi -pedantic -Iinclude -g -c -o obj/othello.o src/othello.c
gcc -Wall -xc -ansi -pedantic -Iinclude -g -c -o obj/board.o src/board.c
gcc -Wall -xc -ansi -pedantic -Iinclude -g -c -o obj/main.o src/main.c
gcc -Wall -xc -ansi -pedantic -Iinclude -g -c -o obj/settings.o src/settings.c
gcc -Wall -xc -ansi -pedantic -Iinclude -g -c -o obj/file.o src/file.c
gcc -Wall -xc -ansi -pedantic -Iinclude -g -c -o obj/minimax.o src/minimax.c
gcc -Wall  -o othello.exe obj/history.o obj/othello.o obj/board.o obj/main.o obj
/settings.o obj/file.o obj/minimax.o
```

Konzole 15: Ukázka sestavení projektu pomocí nástroje `make` na operačním systému **Mac OS X**.

4.2 Spuštění

Pro spuštění programu `othello.exe` je třeba ho nejprve sestavit. Pokud tak nebylo učiněno, následujte prosím sekci 4.1.

V kořenovém adresáři semestrální práce by se měl nacházet soubor jménem `othello.exe`. Tento program je třeba spustit v příkazové konzoli. V operačním systému **Windows** lze využít program **CMD** nebo **PowerShell**. Na **Mac OS X** otevřete aplikaci **Terminal**, která je předinstalovaná se systémem.

Přepínače programu jsou popsány v kompletním zadání úlohy [2]. Zde je vypsána zkrácená verze (Tabulka 1, strana 13). Všechny přepínače jsou *nepovinné* a mohou být zadány v libovolném pořadí. Jejich argumenty se rozdělují na povinné (`<>`) a nepovinné (`[]`). Příklad spuštění programu je vidět v ukázce konzole 16 na straně 13.

Přepínač	Argumenty	Popis
`-pb`	`"<human/minimax>[,name][,d]"`	Popis hráče s černými kameny.
`-pw`	`"<human/minimax>[,name][,d]"`	Popis hráče s bílými kameny.
`-size`	`<r> <c>`	Rozměry hrací plochy.
`-init`	`<r> <c>`	Pozice počátečních kamenů.
`-o`	`"<cesta_k_souboru>"`	Výstupní soubor hry.
`-i`	`"<cesta_k_souboru>"`	Vstupní soubor hry.

Tabulka 1: Popis přepínačů a jejich argumentů.

```
PS C:\semestralka> .\othello.exe -pb minimax -size 6 6 -init 3 3 -o output.txt
(Karel:minimax) > 2_3
(Anna:human) > quit
```

Konzole 16: Ukázka spuštění programu v operačním systému Windows (kvůli limitaci dokumentace nelze ukázat celý výstup).

4.3 Ovládání

Pokud byla hra spuštěna s aspoň jedním lidským hráčem, uživatel má dostupné čtyři příkazy. Tyto příkazy jsou popsány v tabulce 2 na straně 14. Pokud jsou oba hráči nastaveni na `minimax`, pak hra probíhá automaticky dokud nedosáhne konce.

Příkaz	Popis
`save`	Pokud byl program spuštěn s přepínačem `-o`, uživatel může uložit historii hry.
`quit`	Ukončí běh programu a vypíše stav. S přepínačem `-o` se uloží historie hry.
`pass`	Hráč přeskočí svůj tah.
`<r>_<c>`	Uveďte řádek (`<r>`) a sloupec (`<c>`), který chcete hrát.

Tabulka 2: Popis příkazů lidského hráče.

4.4 Struktura vstupního souboru

Vstupní soubor nastavuje vlastnosti hry a může obsahovat historii, která je následně odehrána. Nastavení hry se dá přepsat přepínači při spuštění programu, které jsou popsané v tabulce 1 na straně 13.

Vstupní soubor může obsahovat nastavení popsané v tabulce 3 na straně 14. Ukázkový soubor je vidět v konzolovém výpise 17 na straně 14. Jednotlivé elementy mají stejnou strukturu jako jejich ekvivalentní přepínače a kromě elementu `[game]`, který musí být uveden jako poslední a následován historií hry, mohou být v libovolném pořadí.

Nastavení	Popis
`[pb <desc>]`	Popis hráče s černými kameny. Struktura jako odpovídající přepínač.
`[pw <desc>]`	Popis hráče s bílými kameny. Struktura jako odpovídající přepínač.
`[size <r> <c>]`	Počet řádek (`<r>`) a sloupců (`<c>`) hrací desky.
`[init <r> <c>]`	Nastaví řádek (`<r>`) a sloupec (`<c>`) počátečních kamenů.
`[game]`	Znázorňuje začátek historie hry.

Tabulka 3: Struktura možných nastavení hry pomocí vstupního souboru.

```
user@machine:~/semestralka$ bat input.txt
[size 8 8]
[init 4 4]
[pb minimax]
[game]
3_4
5_3
6_2
2_4
5_6
7_1
1_4
```

Konzole 17: Ukázkový vstupní soubor.

5 Závěr

Věřím, že veškeré požadavky zadání projektu [2] byly splněny, avšak některé z implementací by se stále daly zlepšit. Například, jak bylo zmíněno v sekci 2.3 na straně 5, kontrola tahu při samotném pokusu o něj by byla rychlejší, než počítat všechny možné tahy předem. Zároveň by tato změna ušetřila na paměti, jelikož by nebylo potřeba ukládat výsledek kontroly pro každé herní pole.

Dalším vylepšením by byla evaluační funkce stavu hry. Existují určité pozice, které dávají hráči vyšší taktickou výhodu než jiné. Kvůli dynamické velikosti hrací plochy si ale nejsem jistý, jak by se takový problém řešil obecně.

Zdroje

[1] Sebastian Lague. Algorithms explained - minimax and alpha-beta pruning, 2018.

[2] Ing. František Pártl. Algoritmus minimax v deskové hře othello, 2023.